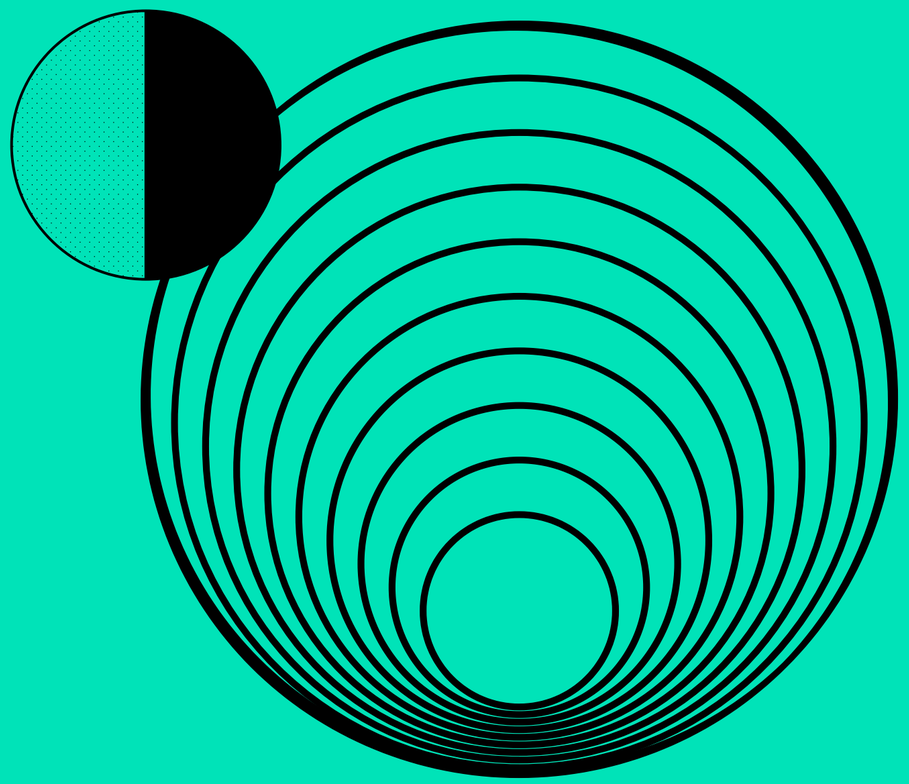
PRIMER ENTREGA SQL

Por: Mora Coghlan
Comisión 89215.

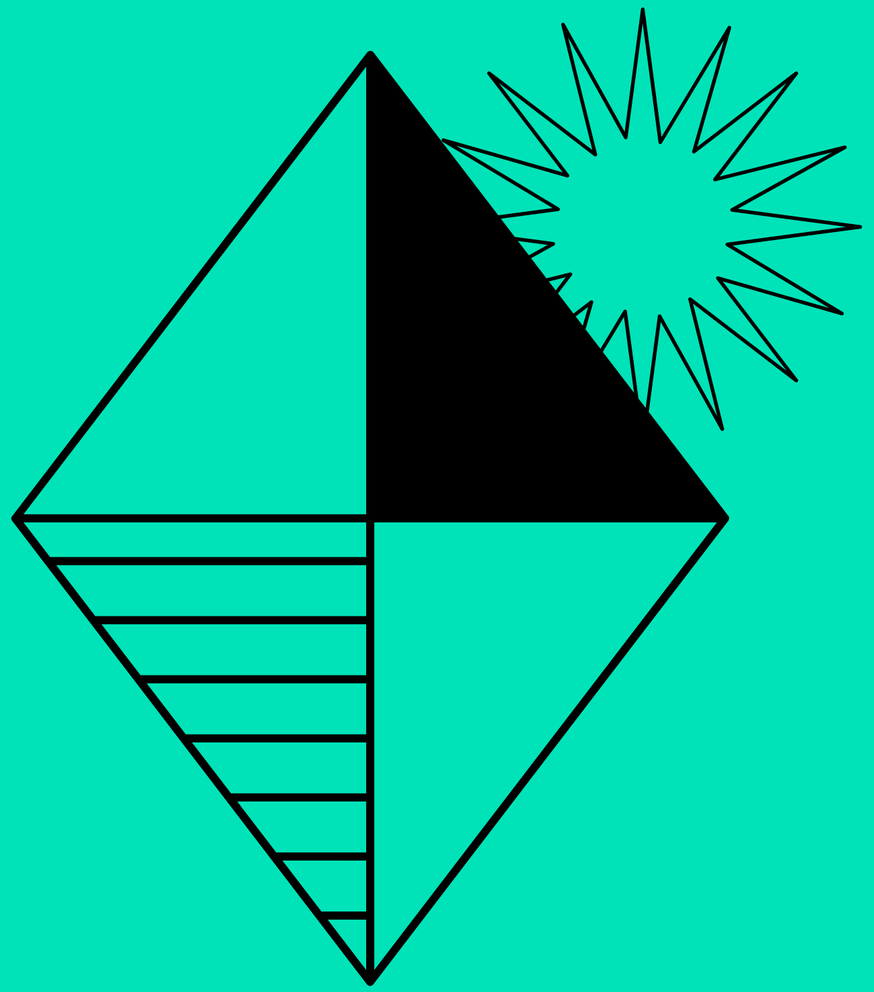


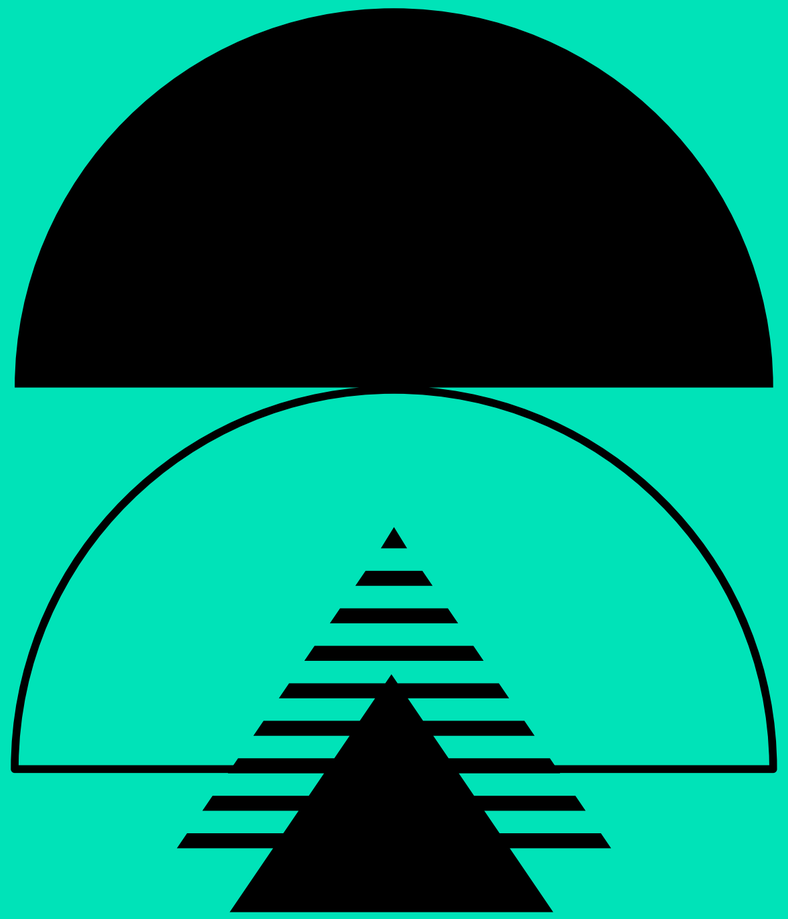
Contenidos.

1. Descripción de la base de datos
2. Diagrama Entidad - Relación
3. Tablas + Campos

Descripción

En este proyecto la base de datos que se llevará a cabo consiste en un emprendimiento de Buzos llamado “cheers lk”. La elección de esta temática se debe a que en mi grupo de amigas, dos de ellas tienen este emprendimiento entonces por llegada a información es más sencillo disponer de una estructura de base de datos realista y detallada en este campo de información.

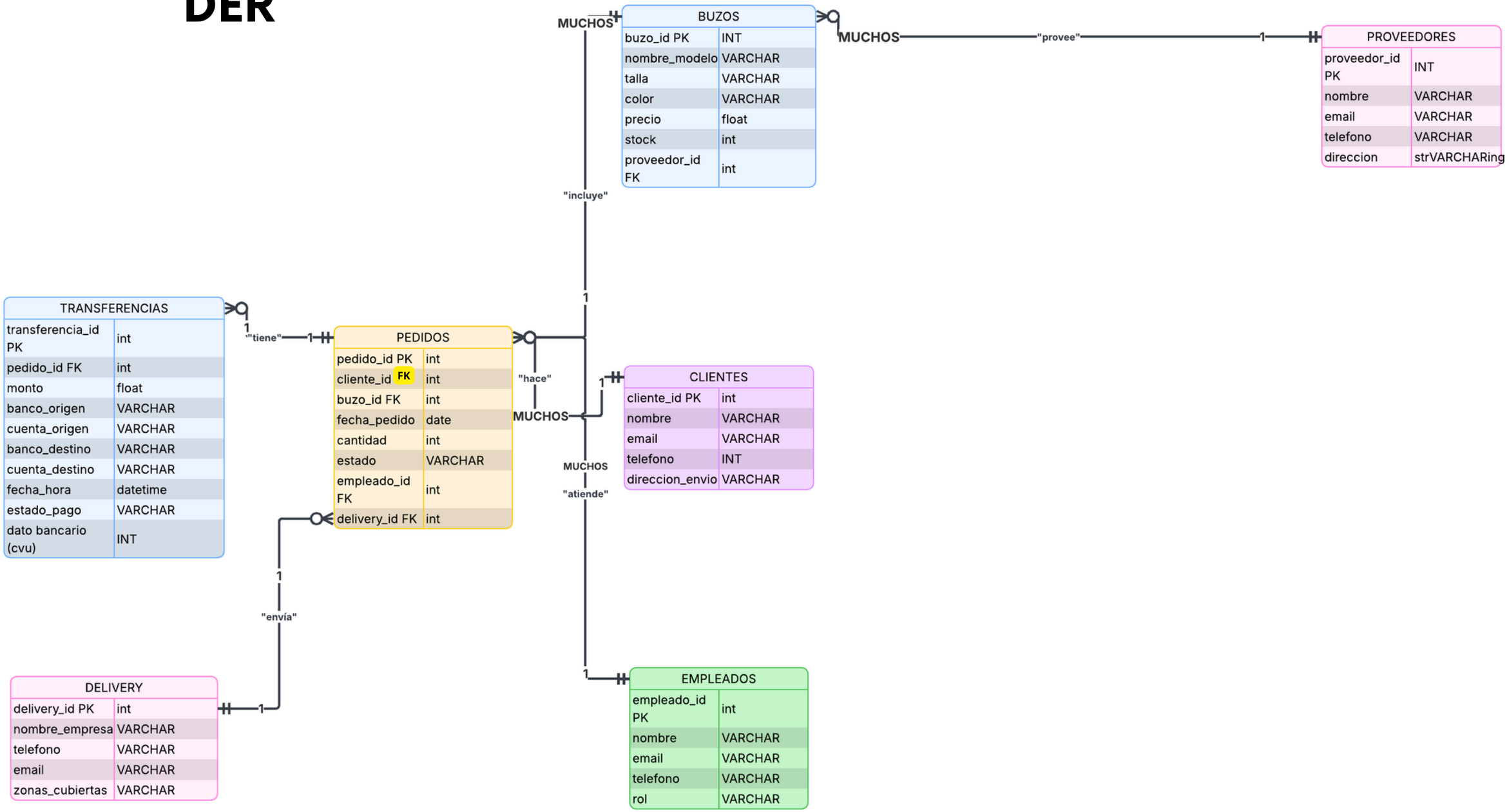




Cheers LK.

El emprendimiento consiste en la venta de buzos personalizados, se diseñan modelos que se publican de los cuales entonces se tiene un menú para elegir el estilo de estampa. El proceso de comercialización consiste en conseguir el material por medio de emprendedores de buzos, a partir de ello estamparlos con la colaboración de un grupo de empleados. Al ser buzos personalizados, se debe tener información del cliente y el nombre del modelo que elige.
Una vez elegido el modelo, se manda a hacer con la transferencia del cliente previa; y se arregla el envío dependiendo la red de deliverys que se pueda contactar para hacer llegar el buzo. Se puede optar por irlo a buscar a algun punto de encuentro.

DER


BUZOS
buzo_id PK INT
nombre_modelo VARCHAR
talla VARCHAR
color VARCHAR
precio float
stock int
proveedor_id FK int
PROVEEDORES
proveedor_id PK INT
nombre VARCHAR
email VARCHAR
telefono VARCHAR
direccion strVARCHARing
MUCHOS
"provee"
1
"incluye"
TRANSFERENCIAS
transferencia_id PK int
pedido_id FK int
monto float
banco_origen VARCHAR
cuenta_origen VARCHAR
banco_destino VARCHAR
cuenta_destino VARCHAR
fecha_hora datetime
estado_pago VARCHAR
dato bancario (cvu) INT
"tiene"
PEDIDOS
pedido_id PK int
cliente_id FK int
buzo_id FK int
fecha_pedido date
cantidad int
estado VARCHAR
empleado_id FK int
delivery_id FK int
"hace"
CLIENTES
cliente_id PK int
nombre VARCHAR
email VARCHAR
telefono INT
direccion_envio VARCHAR
"atiende"
"envía"
DELIVERY
delivery_id PK int
nombre_empresa VARCHAR
telefono VARCHAR
email VARCHAR
zonas_cubiertas VARCHAR
EMPLEADOS
empleado_id PK int
nombre VARCHAR
email VARCHAR
telefono VARCHAR
rol VARCHAR

Aclaración: para mayor comprensión, se le paso la información descrita por la alumna a ChatGPT asi organizaba de manera prolija los datos de cada tabla. Pero los datos y aa disposición de los mismos son idea propia

TABLA: buzos

Funcionalidad:
Almacena la información de los productos que ofrece la empresa (modelos de buzos).
Tipo:
Tabla dimensional / maestra (producto).
Campos:

- buzo_id (PK, INT)
- nombre_modelo (VARCHAR)
- talla (VARCHAR)
- color (VARCHAR)
- precio (FLOAT)
- stock (INT)
- proveedor_id (FK, INT)

Relaciones:

- Muchos a uno con Proveedores
- → Un proveedor puede proveer muchos buzos, pero cada buzo tiene un solo proveedor.
- Muchos a muchos con Pedidos (conceptualmente)
- → Se implementa desde Pedidos (o idealmente con DetallePedido).

Aclaración: para mayor comprensión, se le paso la información descrita por la alumna a ChatGPT asi organizaba de manera prolija los datos de cada tabla. Pero los datos y aa disposición de los mismos son idea propia

TABLA: proveedores

Funcionalidad:
 Guarda los datos de los proveedores que suministran materiales para los buzos.
Tipo:
 Tabla dimensional / maestra.
Campos:
- proveedor_id (PK, INT)
- nombre (VARCHAR)
- email (VARCHAR)
- telefono (VARCHAR)
- direccion (VARCHAR)

Relaciones:
- Uno a muchos con Buzos

Aclaración: para mayor comprensión, se le paso la información descrita por la alumna a ChatGPT asi organizaba de manera prolija los datos de cada tabla. Pero los datos y aa disposición de los mismos son idea propia

TABLA: clientes

Funcionalidad:
Almacena la información de los clientes que realizan pedidos.
Tipo:
Tabla dimensional / maestra.
Campos:

- cliente_id (PK, INT)
- nombre (VARCHAR)
- email (VARCHAR)
- telefono (VARCHAR)
- direccion_envio (VARCHAR)

Relaciones:

- Uno a muchos con Pedidos
- → Un cliente puede realizar varios pedidos.

Aclaración: para mayor comprensión, se le paso la información descrita por la alumna a ChatGPT asi organizaba de manera prolija los datos de cada tabla. Pero los datos y aa disposición de los mismos son idea propia

TABLA: pedidos

Funcionalidad:
Registra cada pedido realizado por un cliente, vinculando productos, empleados, pagos y delivery.
Tipo:
Tabla transaccional / de hechos.
Campos (recomendados):

- pedido_id (PK, INT)
- cliente_id (FK, INT)
- buzo_id (FK, INT)
- empleado_id (FK, INT)
- delivery_id (FK, INT)
- fecha_pedido (DATE)
- cantidad (INT)
- estado (VARCHAR)

Relaciones:

- Muchos a uno con Clientes
- Muchos a uno con Empleados
- Muchos a uno con Delivery
- Muchos a uno con Buzos
- Uno a uno con Transferencias

Aclaración: para mayor comprensión, se le paso la información descrita por la alumna a ChatGPT asi organizaba de manera prolija los datos de cada tabla. Pero los datos y aa disposición de los mismos son idea propia

TABLA: empleados

Funcionalidad:
Contiene los datos de los empleados que se encargan de la estampación de los buzos.
Tipo:
Tabla dimensional / maestra.
Campos:

- empleado_id (PK, INT)
- nombre (VARCHAR)
- email (VARCHAR)
- telefono (VARCHAR)
- rol (VARCHAR)

Relaciones:

- Uno a muchos con Pedidos

Aclaración: para mayor comprensión, se le paso la información descrita por la alumna a ChatGPT asi organizaba de manera prolija los datos de cada tabla. Pero los datos y aa disposición de los mismos son idea propia

TABLA: transferencias

Funcionalidad:
Registra la información bancaria y temporal de los pagos realizados por los pedidos.
Tipo:
Tabla transaccional.
Campos:

- transferencia_id (PK, INT)
- pedido_id (FK, INT)
- monto (FLOAT)
- banco_origen (VARCHAR)
- cuenta_origen (VARCHAR)
- banco_destino (VARCHAR)
- cuenta_destino (VARCHAR)
- fecha_hora (DATETIME)
- estado_pago (VARCHAR)
- cbu (VARCHAR)

Relaciones:

- Uno a uno con Pedidos
- → Cada pedido tiene una única transferencia.

Aclaración: para mayor comprensión, se le paso la información descrita por la alumna a ChatGPT asi organizaba de manera prolija los datos de cada tabla. Pero los datos y aa disposición de los mismos son idea propia

TABLA: delivery

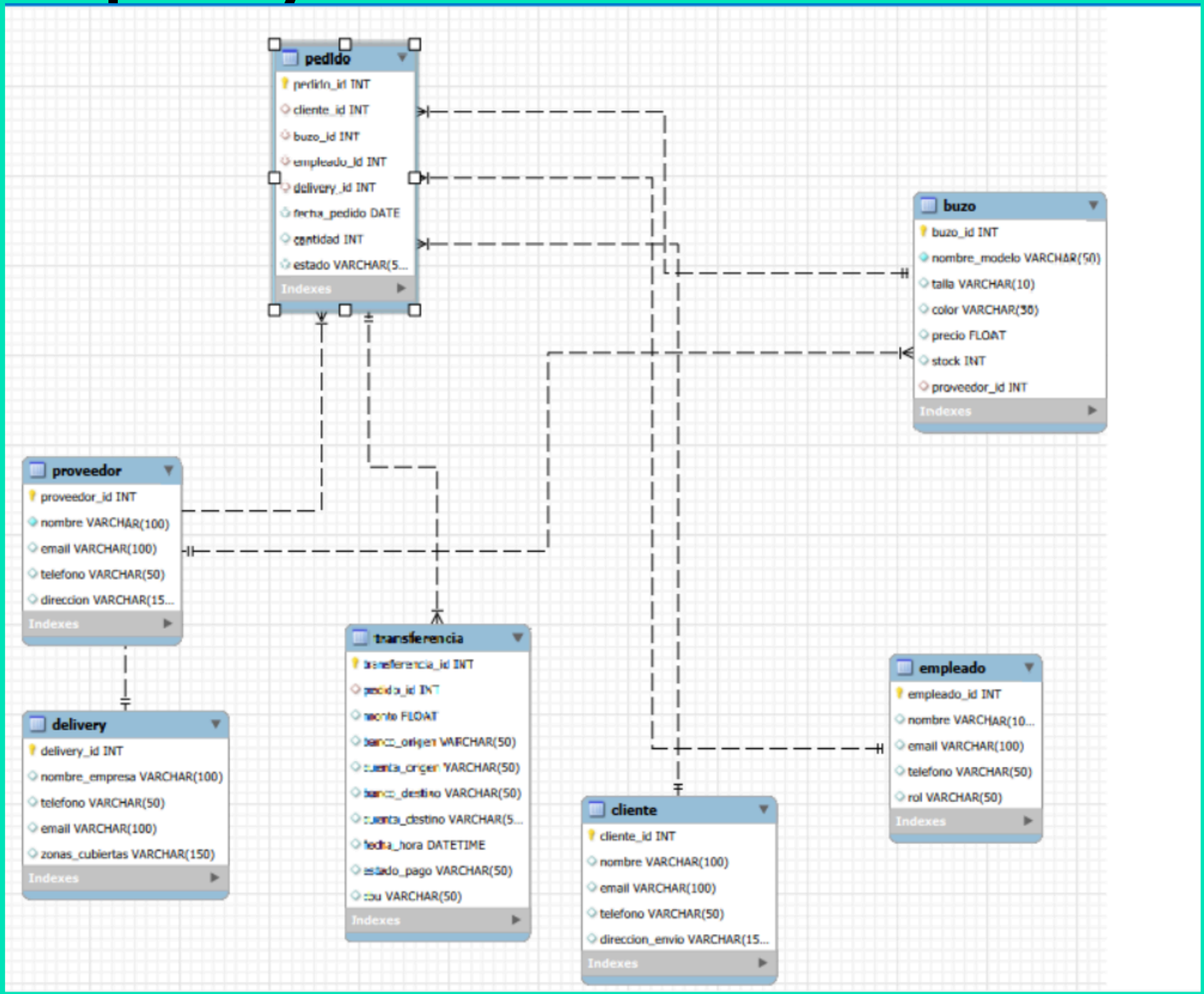
Funcionalidad:
Almacena la información de las empresas encargadas del envío de los pedidos.
Tipo:
Tabla dimensional / maestra.
Campos:

- delivery_id (PK, INT)
- nombre_empresa (VARCHAR)
- telefono (VARCHAR)
- email (VARCHAR)
- zonas_cubiertas (VARCHAR)

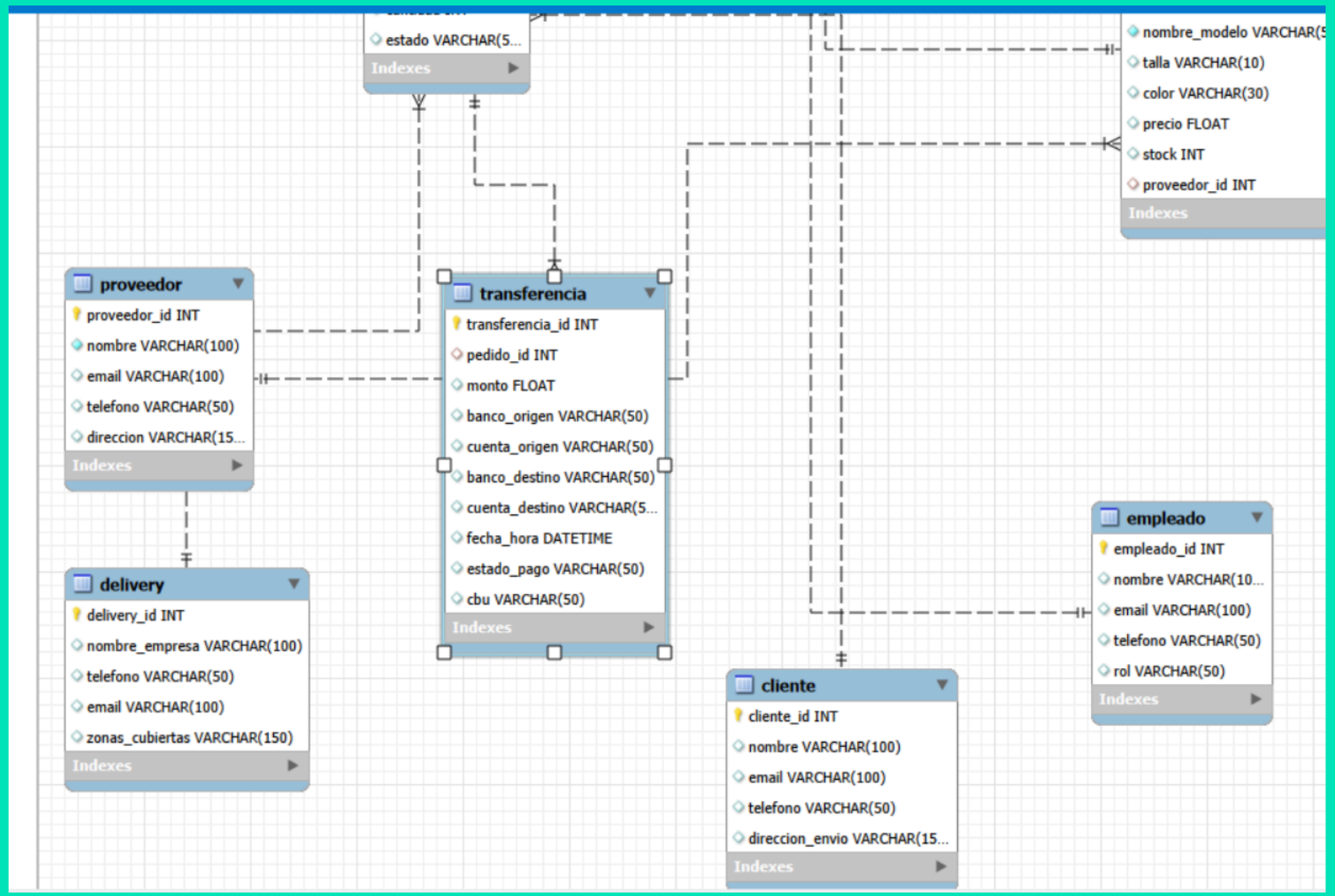
Relaciones:

- Uno a muchos con Pedidos
- → Una empresa de delivery puede entregar muchos pedidos.

DER generado por MySQL



(un poco de zoom por si algo no se lee)



2da entrega: Incerción de datos

Para poder probar el funcionamiento del sistema se insertaron 10 registros de prueba en cada tabla principal.
Las tablas que contienen datos de prueba son:

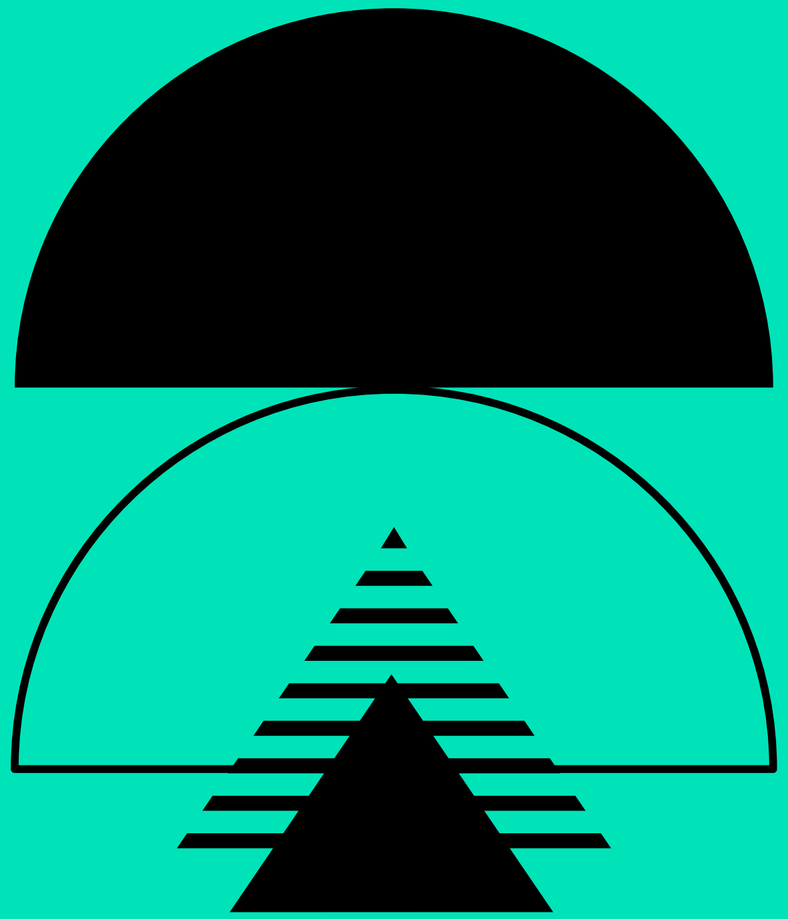
1. proveedor
2. buzo
3. cliente
4. empleado
5. delivery
6. pedido
7. transferencia

Esto ya fue hecho con la herramienta INSERT INTO, para la entrega anterior

Creación de tablas nuevas

Tabla: costos
Se creó una nueva tabla llamada costos para almacenar el costo de producción de cada modelo de buzo.
Objetivo
Separar la información de costos de fabricación de la tabla de productos, permitiendo calcular:

- rentabilidad
- ganancias
- análisis de costos
- Estructura de la tabla
- La tabla almacena:
- id_modelo
- nombre del modelo
- costo de tela
- costo de tinta
- costo de mano de obra
- proveedor
- costo total de producción



3. Cambios en Relaciones (Foreign Keys)

Para mejorar el modelo de datos se realizaron modificaciones en las relaciones entre tablas.

Cambios realizados

1. Se agregó la columna id_modelo en la tabla buzo.
2. Se vinculó la tabla buzo con la tabla costos.

Esto permite que cada buzo esté asociado con su estructura de costos de producción.

Nueva relación creada

buzo.id_modelo → costos.id_modelo

Beneficio

Esto permite:

- calcular ganancias por modelo
- analizar rentabilidad
- separar información comercial de información de producción

VISTAS

Se crearon dos vistas para facilitar consultas frecuentes.

Vista 1 — pedido_pago
Descripción

Esta vista muestra los pedidos que tienen un pago aprobado por transferencia.

Incluye:

- fecha del pedido
- estado del pedido
- cantidad
- estado del pago
- nombre del cliente

Objetivo

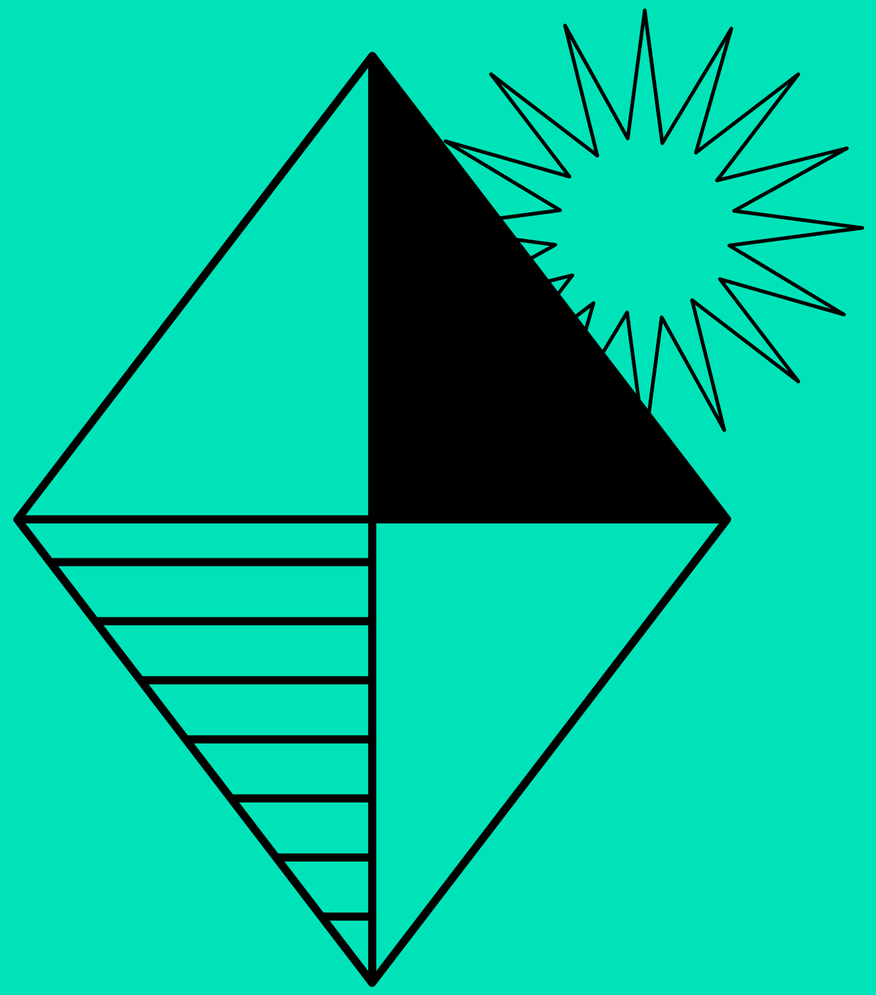
Permitir identificar rápidamente qué pedidos ya fueron pagados correctamente.

Esto facilita tareas como:

- control de pagos
- preparación de envíos
- seguimiento de pedidos confirmados

Tablas utilizadas

- pedido
- transferencia
- cliente



Limit to 1000 rows

```
CREATE VIEW pedido_pago AS (

SELECT pe.fecha_pedido, pe.estado, pe.cantidad, tr.estado_pago, (
SELECT cl.nombre
FROM cheers_lk.cliente AS cl
WHERE pe.cliente_id=cl.cliente_id) AS cliente
FROM cheers_lk.pedido AS pe
LEFT JOIN cheers_lk.transferencia AS tr ON pe.pedido_id=tr.pedido_id
WHERE tr.estado_pago='Aprobado');


SELECT * FROM pedido_pago;
```

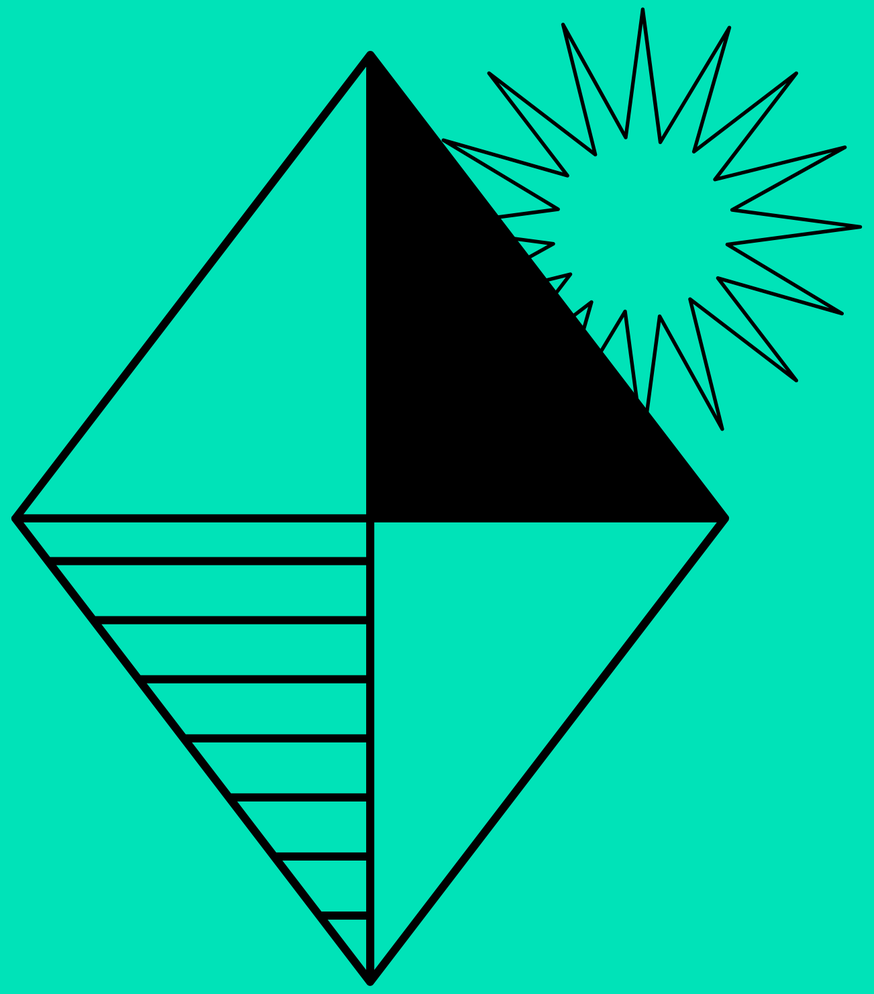
Vista 2 — costo_ganancia Descripción

Esta vista relaciona el costo de producción de un modelo con el precio de venta de los buzos.
También muestra el total de pedidos entregados.
Información que muestra

- costo total de producción
- nombre del modelo
- suma de precios de los buzos
- total de pedidos entregados

Objetivo
Permitir analizar la relación entre costos y ventas, facilitando la evaluación de rentabilidad de los productos.
Tablas utilizadas

- buzo
- costos
- pedido



```sql
CREATE VIEW cheers_lk.costo_ganancia AS (

SELECT
cs.total_costo,
cs.nombre_modelo,
SUM(bu.precio) AS total_precio_buzos,
(
    SELECT SUM(pe.cantidad)
    FROM cheers_lk.pedido AS pe
    WHERE pe.estado = 'Entregado'
) AS total_pedidos_entregados
FROM cheers_lk.buzo AS bu
LEFT JOIN cheers_lk.costos AS cs
ON bu.id_modelo = cs.id_modelo
GROUP BY cs.id_modelo, cs.total_costo, cs.nombre_modelo
);

SELECT * FROM cheers_lk.costo_ganancia;
```

FUNCIONES

Función: ganancia_por_modelo Descripción

Esta función calcula la ganancia promedio de un modelo de buzo.
La ganancia se obtiene restando:

precio promedio de venta - costo de producción

Objetivo
Permitir conocer la rentabilidad de cada modelo de buzo de forma rápida.
Esta función puede utilizarse para:

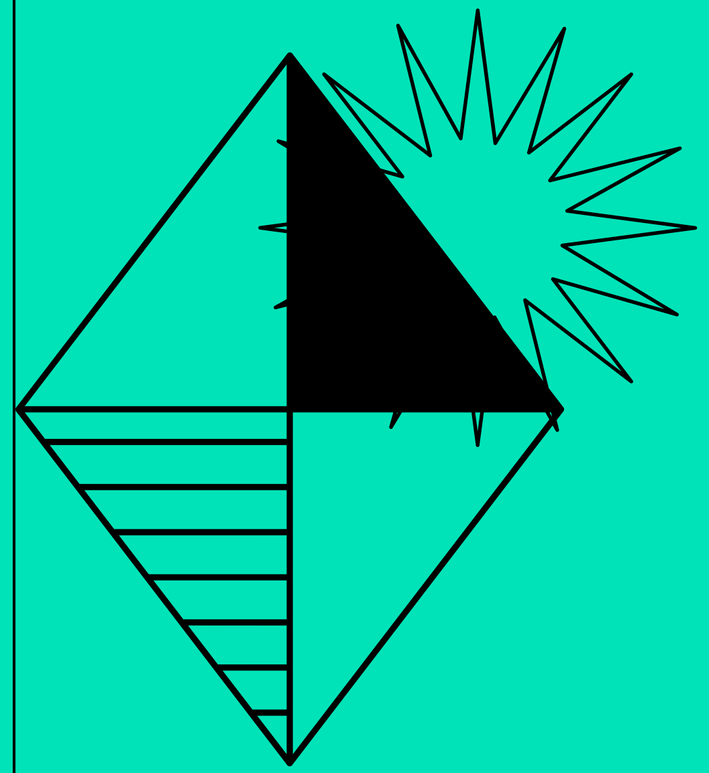
- análisis financiero
- decisiones de precios
- evaluación de costos

Tablas utilizadas

- buzo
- costos

Datos utilizados

- buzo.precio
- costos.total_costo
- id_modelo



PROCEDURES

Procedure: pedidos_por_cliente
Descripción

Este procedimiento permite consultar todos los pedidos realizados por un cliente específico.

Devuelve información como:

- ID del pedido
- nombre del cliente
- buzo comprado
- fecha del pedido
- cantidad
- estado del pedido

Objetivo

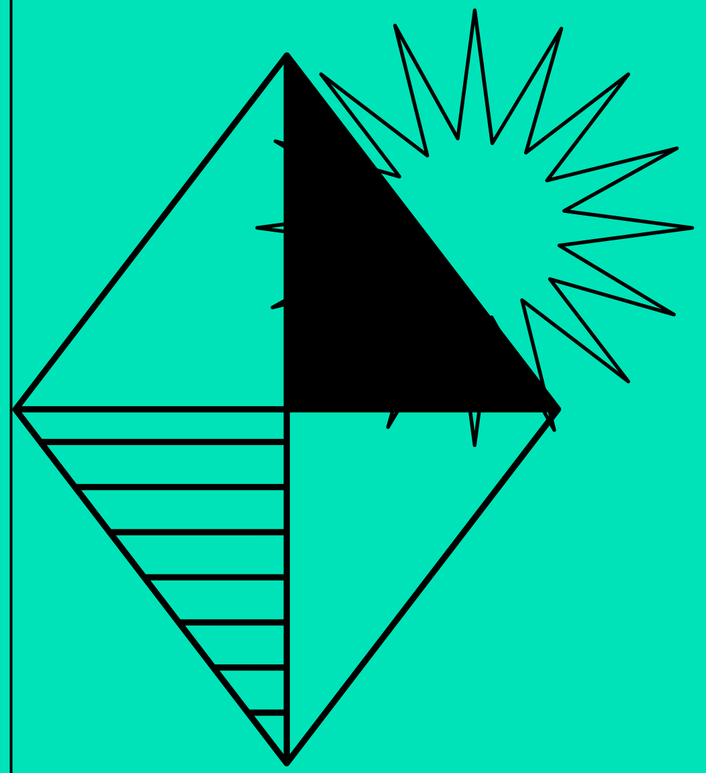
Simplificar la consulta del historial de pedidos de un cliente sin tener que escribir la consulta manualmente cada vez.

Beneficio

Mejora la organización de las consultas dentro del sistema y facilita el acceso a la información de ventas.

Tablas utilizadas

- pedido
- cliente



TRIGGERS

Trigger: actualizar_stock
Descripción

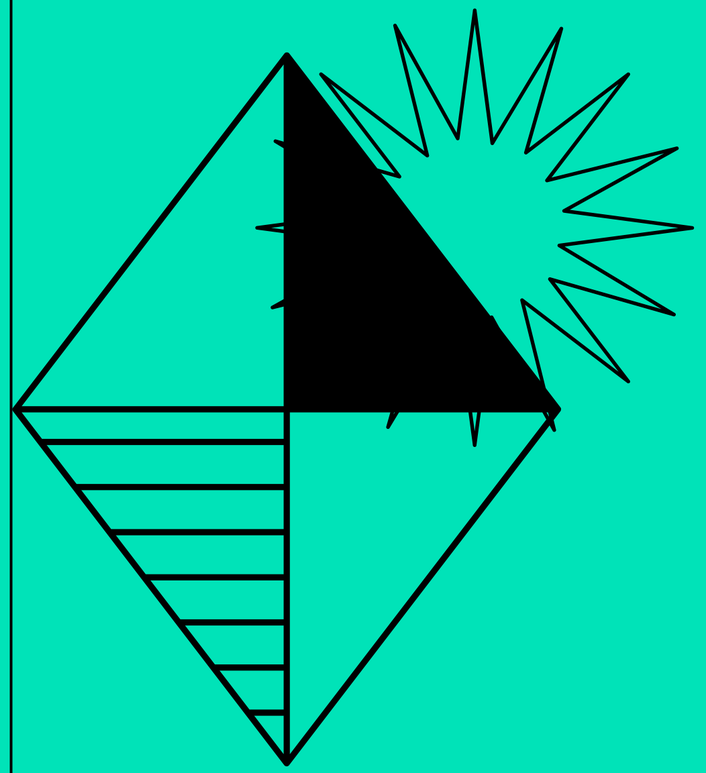
Este trigger se ejecuta automáticamente cuando se inserta un nuevo pedido en la tabla pedido.

Objetivo

Actualizar automáticamente el stock del producto vendido.

Beneficio

Evita tener que actualizar manualmente el stock cada vez que se registra un pedido.

Tablas afectadas

- pedido
- buzo